

Génie Télécommunications et Réseaux

Projet N°1 de Cybersécurité et DevSecOps

Sécurisation complète d'une image Docker + SBOM

Signature (SLSA 3)



Réalisé par :

MARCHOUAL Houssameddine

EL ABBADY Wassima

Encadré par :

Pr. Mohammed AIRAJ

Année universitaire : 2025 – 2026

Table des matières

Introduction générale	2
I. Introduction	3
1.1. Contexte du projet et enjeux DevSecOps	3
1.2. Objectifs de sécurisation d'une image Docker	3
1.3. Présentation de SLSA 3 et de la chaîne de confiance	5
II. Prérequis et environnement	7
2.1. Environnements utilisés	7
2.1.1. Windows 11 + WSL (Ubuntu).....	7
2.1.2. Docker Desktop	8
2.1.3. Python	9
2.1.4. BuildKit, Syft, Cosign, Gype	10
2.2. Configuration initiale	12
2.3. Authentifications (Docker Hub, Cosign keyless)	13
2.3.1. Docker Hub	13
2.3.2. Cosign keyless	14
2.3.3. Sigstore	14
III. Développement de l'application	16
3.1. Création de app.py	16
3.2. Fonctionnalités de l'application	17
3.3. Exécution et test local	17
IV. Construction de l'image Docker	18
4.1. Dockerfile multi-stage build	18
4.2. Choix de l'image distroless	20
4.3. Création d'un utilisateur non-root	23
4.4. Optimisation de la taille finale	24

V. Publication de l'image sur Docker Hub	25
5.1. Tag de l'image locale	25
5.2. Push vers Docker Hub	25
5.3. Récupération du digest	26
VI. Génération du SBOM	27
6.1. Rôle et utilité d'un SBOM	27
6.2. Génération avec Syft (sbom.json, sbom_summary.json)	28
6.3. Vérification du SBOM	29
VII. Analyse de sécurité de l'image	30
7.1. Scan avec Gype (mode offline)	30
7.2. Vulnérabilités détectées	32
7.3. Gestion et interprétation des résultats	33
VIII. Signature de l'image Docker	34
8.1. Présentation de la signature keyless (OIDC)	34
8.2. Connexion Cosign / Docker Hub	34
8.3. Signature de l'image	35
8.4. Vérification de la signature (SLSA 3 compliant)	36
IX. Conclusion	37
9.1. Résumé des étapes	37
9.2. Avantages du modèle DevSecOps et pistes d'améliorations	38

Introduction générale

Avec la montée en puissance des technologies conteneurisées et la généralisation des pratiques DevSecOps, la sécurité des applications et des images Docker est devenue un enjeu majeur pour les entreprises et les développeurs. La conteneurisation offre de nombreux avantages, tels que la portabilité, la scalabilité et la simplification des déploiements, mais elle introduit également de nouveaux vecteurs de vulnérabilités.

Dans ce contexte, la sécurisation des images Docker à toutes les étapes du cycle de vie est essentielle pour prévenir les attaques, garantir l'intégrité des logiciels et se conformer aux standards de sécurité modernes, tels que SLSA 3. La génération d'un SBOM (Software Bill of Materials), le scan des vulnérabilités, ainsi que la signature sécurisée des images sont des pratiques indispensables pour assurer la confiance dans les conteneurs déployés en production.

Ce rapport présente le projet de sécurisation complète d'une image Docker Python en utilisant une image distroless, avec génération de SBOM, analyse des vulnérabilités, et signature keyless via Cosign, tout en respectant les bonnes pratiques DevSecOps 2025. Un exemple pratique d'attaque a également été intégré afin de démontrer la robustesse et l'efficacité des mesures de sécurité mises en place.



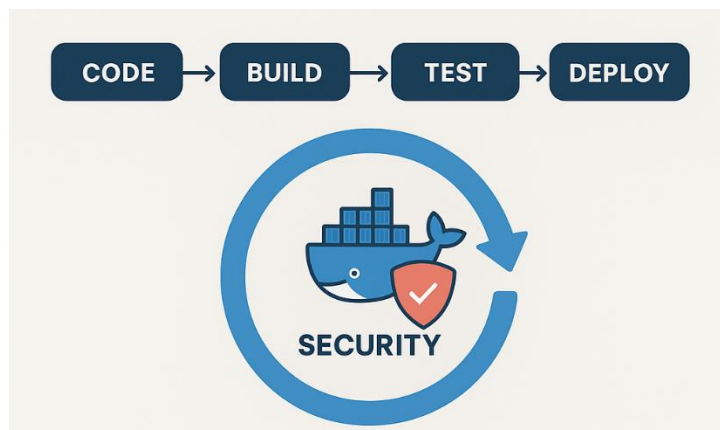
I. Introduction

1.1. Contexte du projet et enjeux DevSecOps

Avec la popularité croissante des conteneurs, les entreprises adoptent de plus en plus des pipelines DevSecOps, où la sécurité est intégrée dès la phase de développement. Les images Docker, bien qu'efficaces pour déployer des applications, présentent des risques :

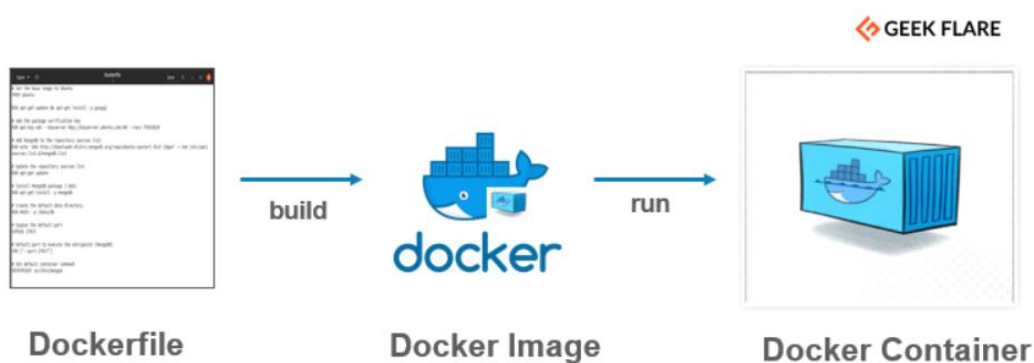
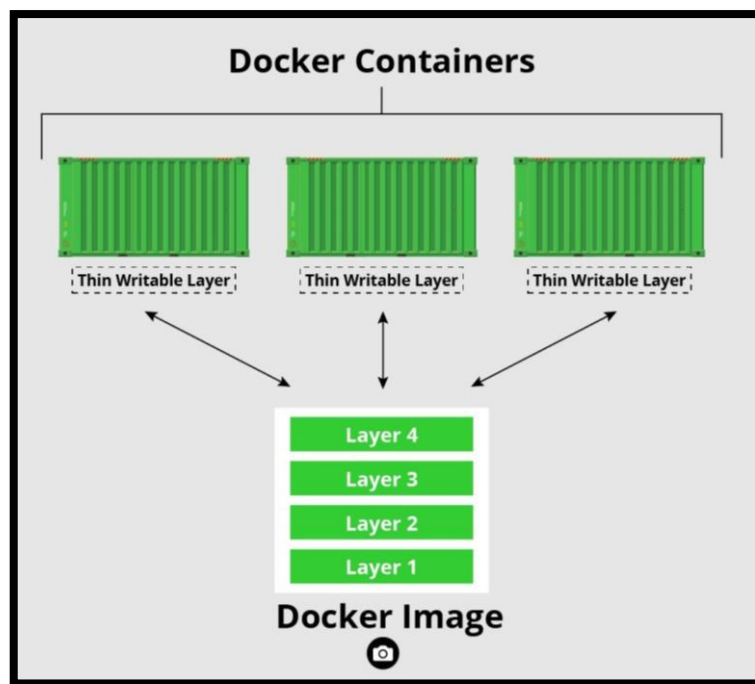
- Vulnérabilités dans les dépendances logicielles.
- Images contenant des privilèges inutiles (root).
- Risques liés à l'intégrité et à l'authenticité des images dans les registres publics.

Le DevSecOps vise à intégrer des pratiques de sécurité automatisées dans le cycle de vie logiciel : construction, test, déploiement et maintenance. Dans ce contexte, sécuriser une image Docker est un exemple concret de l'application des principes DevSecOps.



1.2. Objectifs de sécurisation d'une image Docker

- Image Docker : un package contenant tout le nécessaire pour exécuter une application (code, bibliothèques, fichiers de configuration).
- Conteneur Docker : une instance en cours d'exécution d'une image Docker.
- Image classique (base image) : image de base utilisée pour construire d'autres images, souvent un OS léger comme Debian ou Alpine.
- Image distroless : image minimale contenant uniquement les fichiers nécessaires à l'exécution de l'application, sans shell ni utilitaires superflus, réduisant ainsi la surface d'attaque.



Les objectifs principaux du projet sont :

- **Créer une image Python sécurisée avec multi-stage build** : séparer la compilation et l'exécution pour réduire la taille et les dépendances.
- **Utiliser une image distroless** : éliminer les composants inutiles et minimiser les vulnérabilités.
- **Générer un SBOM** : lister toutes les dépendances logicielles pour faciliter la vérification et la gestion des risques.
- **Scanner l'image avec Gype ou Trivy** : identifier les vulnérabilités critiques avant déploiement.
- **Signer l'image avec Cosign keyless** : garantir l'authenticité et l'intégrité de l'image selon la norme SLSA 3.

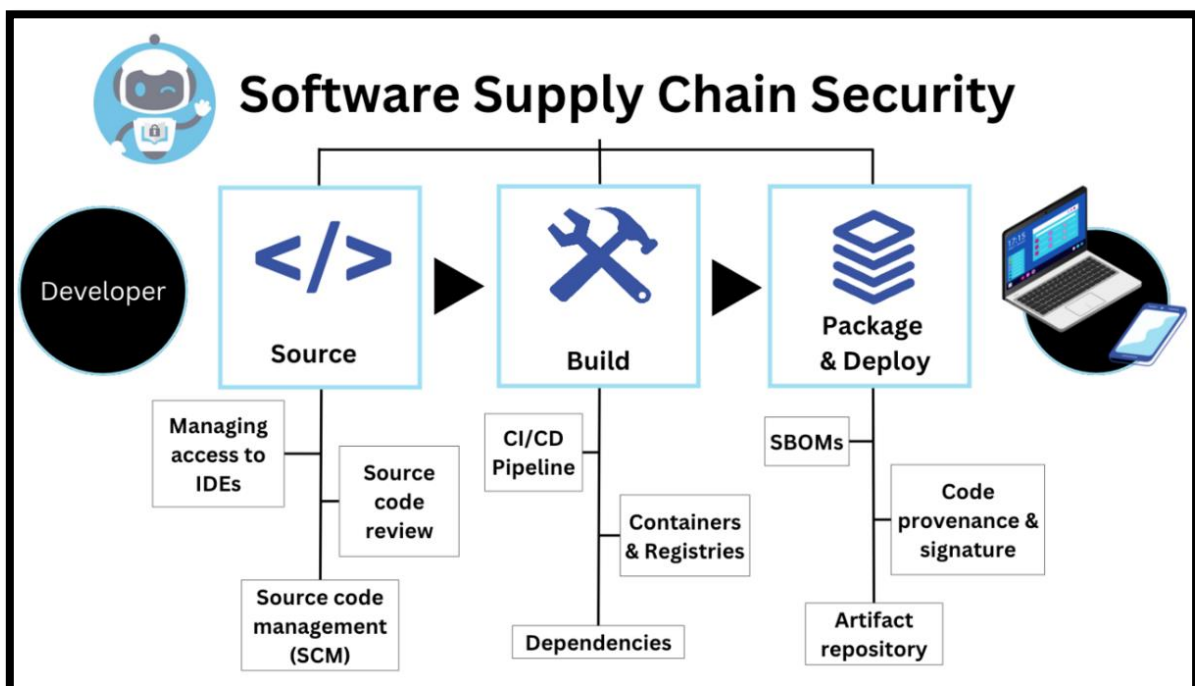
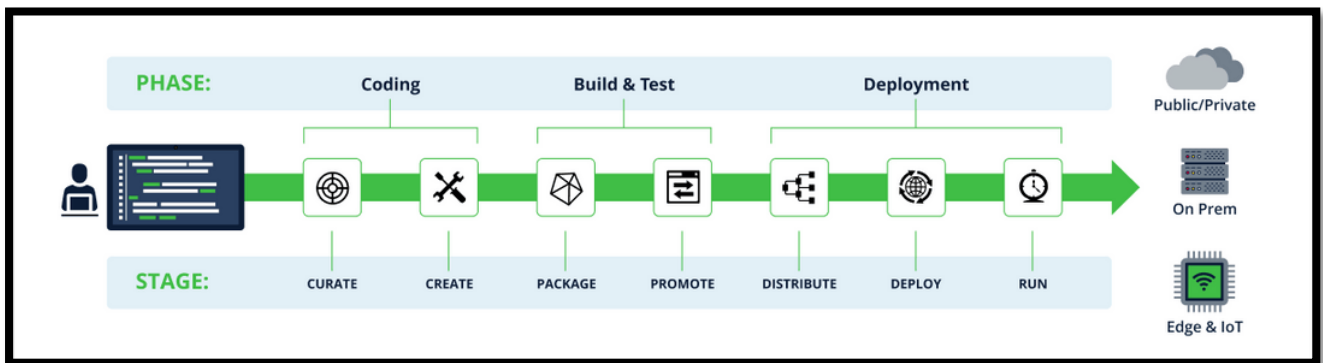
1.3. Présentation de SLSA 3 et de la chaîne de confiance

Supply chain logicielle :

La supply chain *logicielle* ou chaîne d'approvisionnement logicielle désigne l'ensemble des étapes, outils et processus impliqués dans le développement, la construction, la distribution et le déploiement d'un logiciel.

Elle inclut : le code source, les dépendances externes, les systèmes de build, les artefacts (images Docker, binaires), ainsi que leur distribution aux utilisateurs finaux.

L'objectif principal est de gérer et sécuriser tout le cycle de vie du logiciel, afin de réduire les risques d'introduction de vulnérabilités ou de composants compromis.



Chaîne de confiance (trust chain):

La *chaîne de confiance* est un mécanisme de sécurité qui permet de garantir l'authenticité et l'intégrité de chaque composant au sein de la supply chain logicielle.

Elle s'appuie sur des outils et pratiques tels que :

- La génération de SBOM (Software Bill Of Materials),
- La signature des artefacts avec Cosign ou in-toto,
- La vérification de l'intégrité et de la provenance des images et binaires.

Ainsi, la chaîne de confiance assure que chaque artefact utilisé dans la supply chain est fiable et n'a pas été altéré, renforçant la sécurité globale du processus de développement.

SLSA 3 (Supply-chain Levels for Software Artifacts) :

SLSA est un référentiel de bonnes pratiques pour sécuriser la supply chain logicielle. Le niveau SLSA 3 garantit que :

Les artefacts sont construits dans un environnement contrôlé et reproductible,

- Chaque étape du build est traçable et auditable,
- Les artefacts peuvent être signés et vérifiés dans une chaîne de confiance.

En combinant SLSA 3 et la chaîne de confiance, on obtient une supply chain logicielle sécurisée, résiliente aux attaques et conforme aux standards DevSecOps modernes.



II. Prérequis et environnement :

2.1. Environnements utilisés

2.1.1. Windows 11 + WSL (Ubuntu)

Pour réaliser ce projet, nous avons utilisé Windows 11 comme système d'exploitation hôte.

Afin de bénéficier d'un environnement Linux complet pour Docker et Python, nous avons activé WSL (Windows Subsystem for Linux) et installé Ubuntu 24.04.3 LTS.

Avantages de WSL pour le projet :

- Permet d'exécuter des commandes Linux directement depuis Windows.
- Compatible avec Docker Desktop pour Windows.
- Réduit les problèmes de compatibilité entre outils DevOps et Windows.

```
PS C:\WINDOWS\system32> wsl -d ubuntu
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Sat Dec 6 02:10:08 +01 2025

System load:  0.02               Processes:            48
Usage of /:   0.5% of 1006.85GB  Users logged in:     2
Memory usage: 27%               IPv4 address for eth0: 172.28.137.27
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/user123/.hushlogin file.
user123@HDM1:/mnt/c/WINDOWS/system32$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 24.04.3 LTS
Release:      24.04
Codename:     noble
user123@HDM1:/mnt/c/WINDOWS/system32$
```

2.1.2. Docker Desktop :

Docker Desktop est l'outil principal pour la gestion des conteneurs.

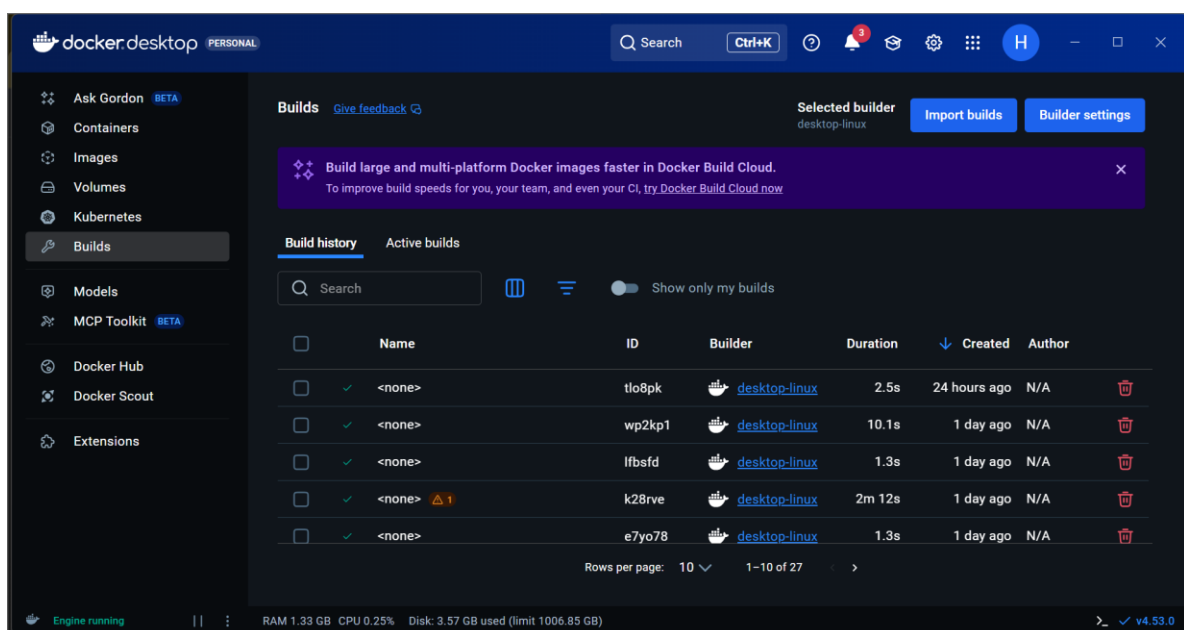
Il permet :

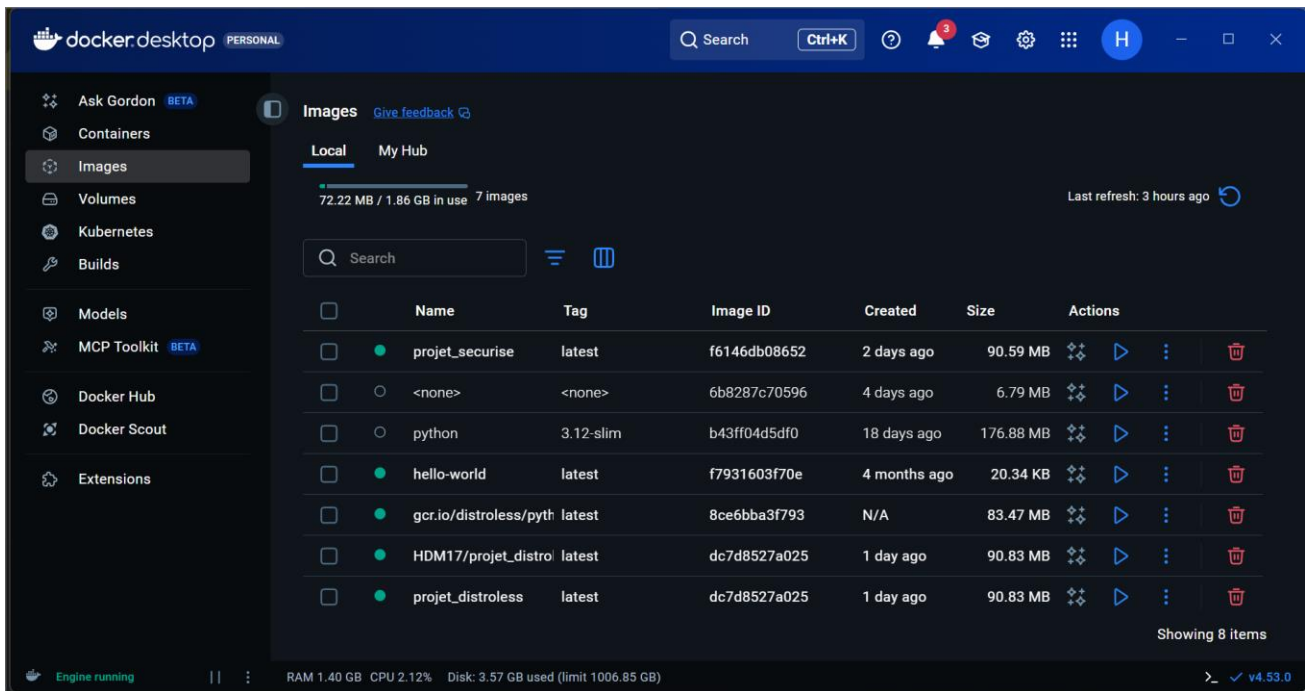
- La construction d'images Docker avec multi-stage build,
- L'exécution de conteneurs en environnement isolé,
- L'intégration avec WSL pour les builds Linux-native.

Fonctionnalités utilisées dans le projet :

- Images distroless pour des images légères et sécurisées.
- Tagging et push vers Docker Hub pour partager l'image.
- Connexion à Cosign keyless pour la signature d'images.

```
user123@HDM1:/mnt/c/WINDOWS/system32$ docker info | grep -E "Server Version|Storage Driver|Operating System"
Server Version: 29.0.1
Storage Driver: overlayfs
Operating System: Docker Desktop
user123@HDM1:/mnt/c/WINDOWS/system32$ docker info | grep -E "Client:|Server:|Containers:| Running:| Paused:| Stopped
er:"
Client:
Server:
Containers: 20
Running: 0
Paused: 0
Stopped: 20
Images: 7
Server Version: 29.0.1
Storage Driver: overlayfs
Operating System: Docker Desktop
user123@HDM1:/mnt/c/WINDOWS/system32$
```





2.1.3. Python

Le projet repose sur une application Python simple pour démontrer la construction et sécurisation d'une image Docker.

Version utilisée : Python 3.11

Rôles de Python dans le projet :

- Contenu de l'application principale (fichier app.py)
- Dépendances gérées via requirements.txt
- Compatibilité avec Docker multi-stage build



```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker run --rm hdm17/projet_distroless:latest --version
Python 3.11.2
```

C'est le Python qui est à l'intérieur de ton image Docker distroless → version 3.11.2

```
user123@HDM1:/mnt/c/WINDOWS/system32$ python3 --version
Python 3.12.3
```

C'est le Python installé dans ton Ubuntu WSL → version 3.12.3

2.1.4. BuildKit, Syft, Cosign, Gype

Ces outils sont essentiels pour sécuriser l'image Docker et générer la SBOM :

- BuildKit : Permet des builds Docker plus rapides et optimisés.

```
user123@HDM1:/mnt/c/WINDOWS/system32$ docker buildx ls
NAME/NODE      DRIVER/ENDPOINT  STATUS  BUILDKIT  PLATFORMS
default*       docker          running v0.25.2   linux/amd64 (+3)
\_ default     \_ default      error
desktop-linux
```



- Syft : Génère un SBOM (Software Bill Of Materials) complet.

```
user123@HDM1:/mnt/c/WINDOWS/system32$ syft --version
syft 1.38.0
```



- Cosign : Outil pour signer et vérifier les images Docker (support keyless via OIDC).

```
user123@HDM1:/mnt/c/WINDOWS/system32$ cosign version  
COSIGN  
cosign: A tool for Container Signing, Verification and Storage in an OCI registry.  
  
GitVersion:    v3.0.2  
GitCommit:     84449696f0658a5ef5f2abba87fdd3f8b17ca1be  
GitTreeState:  clean  
BuildDate:     2025-10-10T18:17:56Z  
GoVersion:     go1.25.1  
Compiler:      gc  
Platform:      linux/amd64
```



- Grype : Scanner de vulnérabilités pour les images Docker.

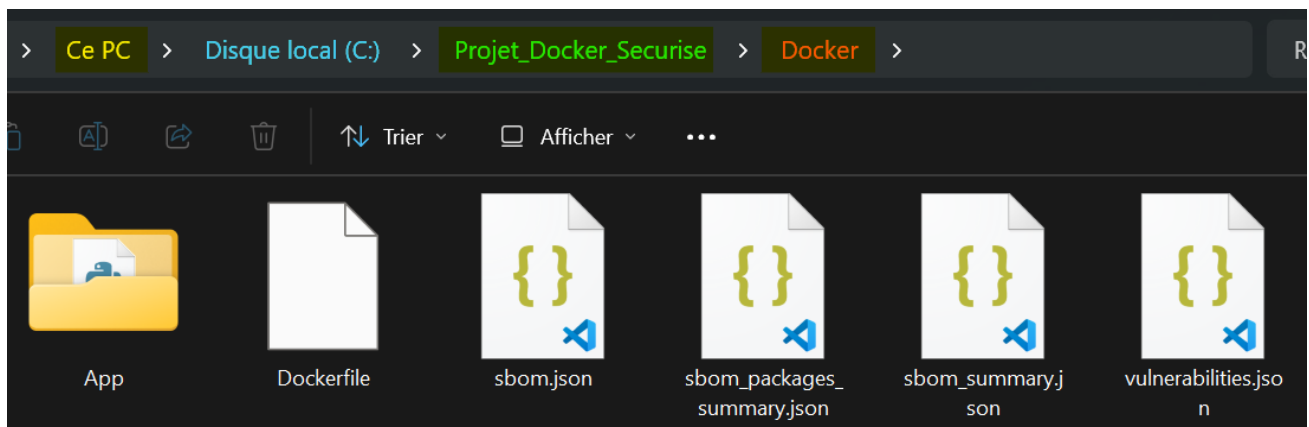
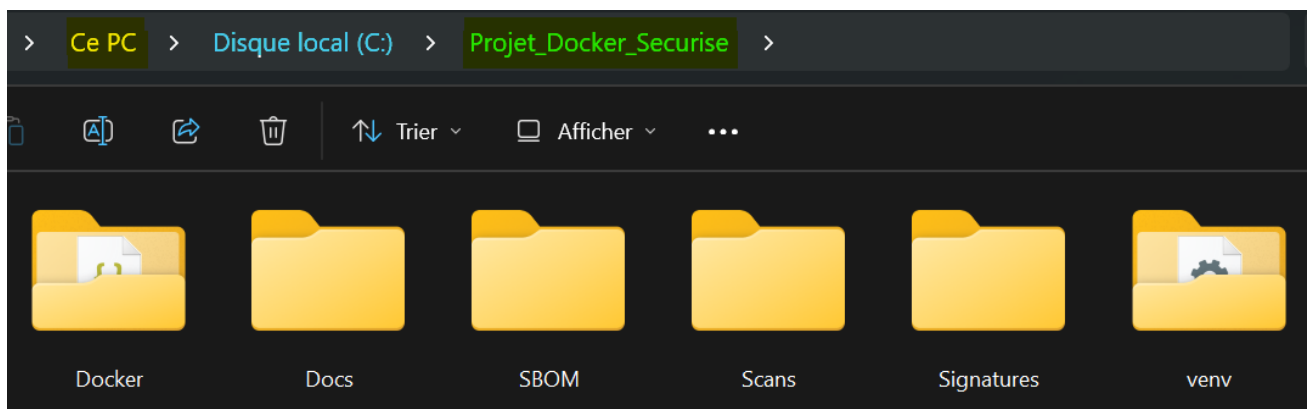
```
user123@HDM1:/mnt/c/WINDOWS/system32$ grype --version  
grype 0.104.1
```



2.2. Configuration initiale

Avant de démarrer le projet, certaines configurations sont nécessaires :

- Création d'un dossier projet dans WSL : `/mnt/c/Projet_Docker_Securise`
- Installation de [Docker Desktop](#) et vérification de la compatibilité [WSL](#)
- Vérification des versions [Python](#), [BuildKit](#), [Syft](#), [Cosign](#), et [Grype](#)
- Création des fichiers essentiels pour le projet :
 - [Dockerfile](#) pour le multi-stage build
 - [app.py](#) pour l'application Python
 - [requirements.txt](#) pour les dépendances



2.3. Authentications (Docker Hub, Cosign keyless)

2.3.1. Docker Hub

Docker Hub est une plateforme en ligne officielle fournie par Docker Inc. Elle sert de registre (registry) permettant :

- d'héberger des images Docker (publiques ou privées),
- de partager ces images avec d'autres utilisateurs,
- d'automatiser le déploiement via des intégrations CI/CD,
- d'accéder à des images officielles, vérifiées ou communautaires.

Docker Hub est un dépôt central où les développeurs stockent, distribuent et téléchargent des images Docker.

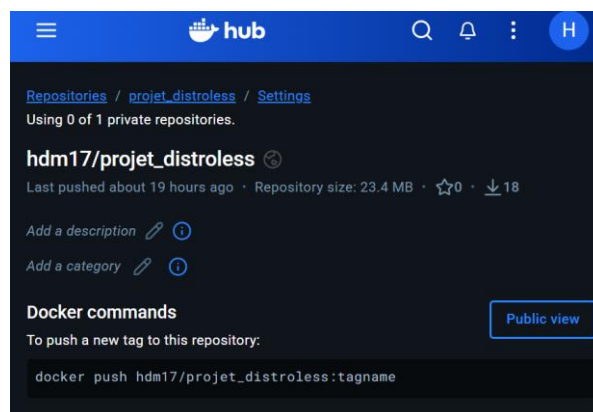


➤ Connexion au compte Docker Hub pour push/pull des images.

```
user123@HDM1:/mnt/c/WINDOWS/system32$ docker login
Authenticating with existing credentials... [Username: hdm17]

Info → To Login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```



2.3.2. Cosign keyless

Cosign Keyless est un mode de signature d'images Docker (ou autres artefacts) qui permet de signer sans créer, gérer ou stocker de clés cryptographiques.

Il utilise une authentification OIDC (Google, GitHub, Microsoft, etc.) pour générer automatiquement une clé temporaire gérée par le service Fulcio et enregistrée dans Rekor (transparency log).

- On signe une image sans clé privée
- Cosign génère une clé éphémère automatiquement
- La signature est enregistrée publiquement dans Rekor
- N'importe qui peut vérifier l'authenticité de ton image

2.3.3. Sigstore

Sigstore est un projet open-source créé par Google, Red Hat et la Linux Foundation.

Il fournit des services pour :

- Signer des artefacts (images Docker, binaires, SBOM, etc.)
- Vérifier des signatures
- Gérer des certificats éphémères
- Maintenir un registre public des signatures

Il est composé de trois briques principales :

Fulcio

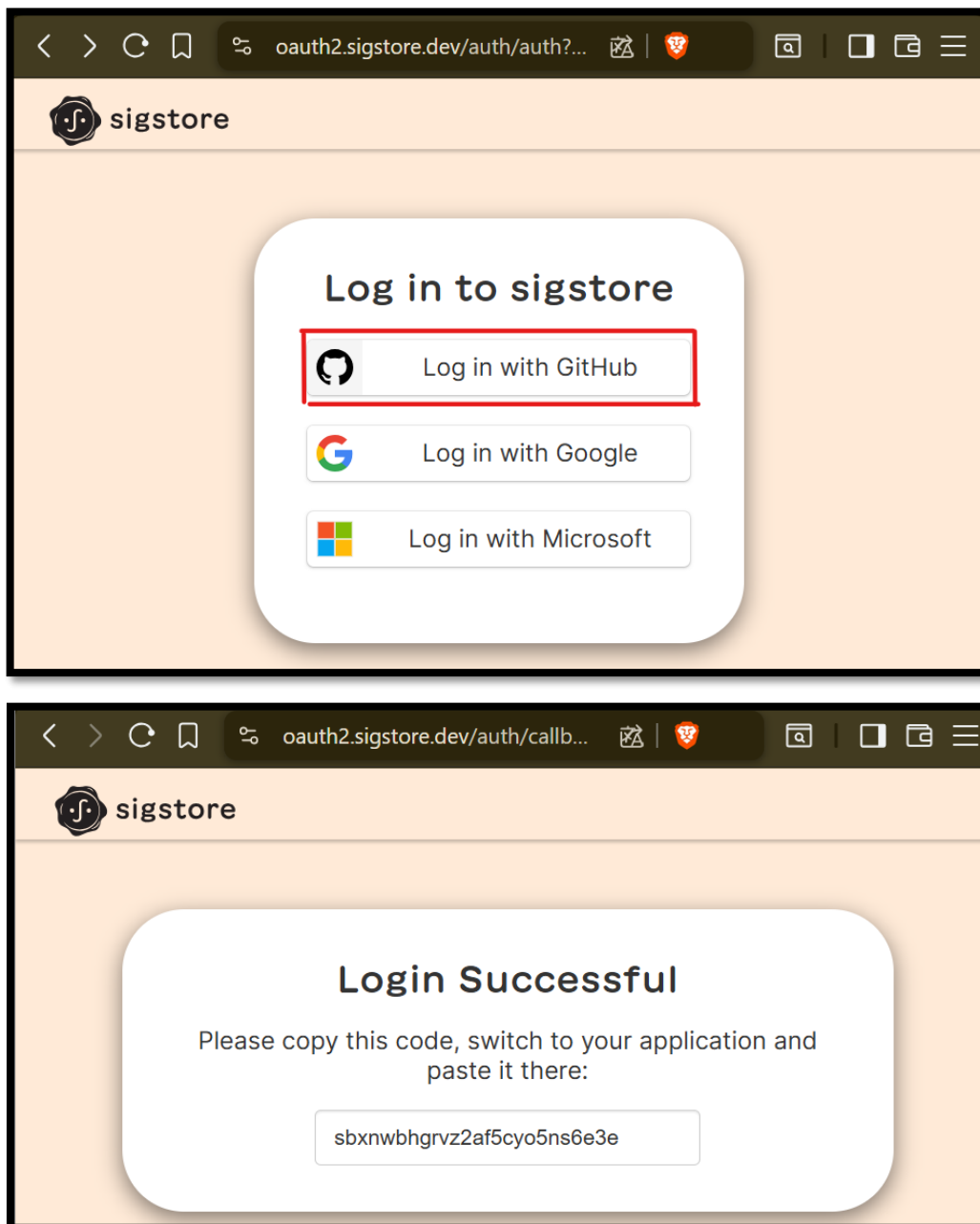
Autorité de certification qui génère des certificats éphémères basés sur OIDC (Google, GitHub, Microsoft...).

Rekor

- Transparency log public où sont enregistrées
- Les signatures
- Les certificats
- Les preuves cryptographiques

Cosign

Un outil en ligne de commande permettant de signer, vérifier et attacher des signatures à des images ou fichiers.



III. Développement de l'application

3.1. Création de app.py :

L'application est développée en Python 3.12.

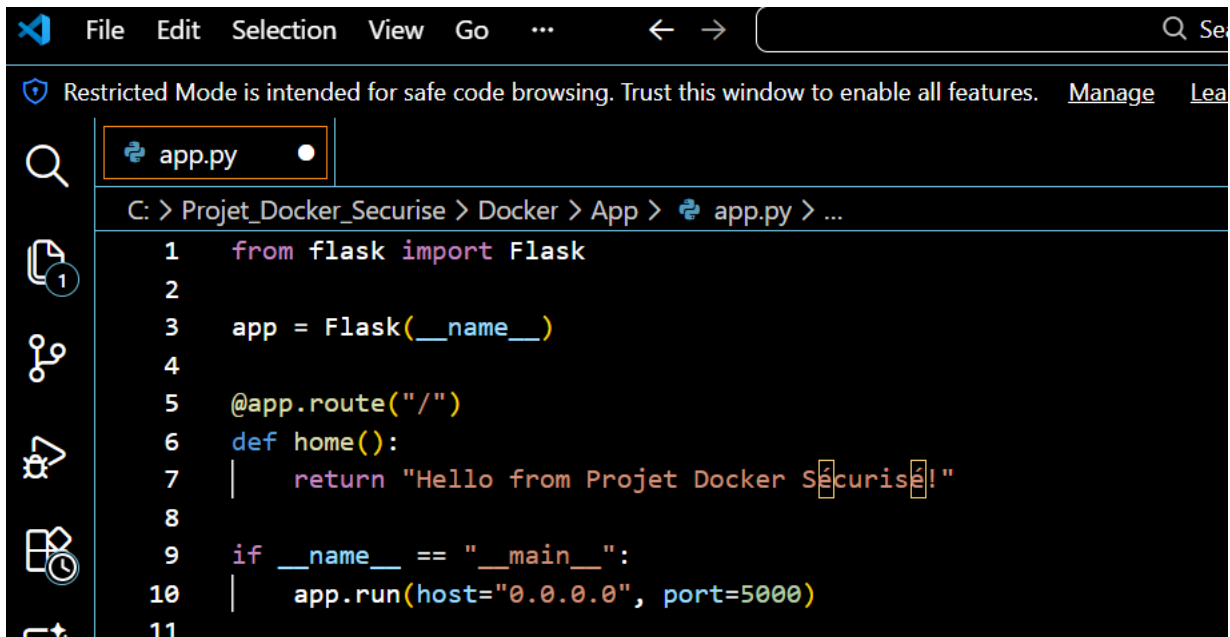
Elle constitue le cœur du projet Docker sécurisé.

L'objectif est de construire une image Docker minimaliste (distroless) contenant uniquement le nécessaire pour exécuter l'application.

Flask est un micro-framework web en Python. C'est léger, simple et très utilisé pour créer des applications web rapidement, des APIs, ou des services backend.

- Micro-framework : Flask fournit juste l'essentiel pour créer des routes, gérer des requêtes HTTP et rendre des templates HTML. Pas de fonctionnalités lourdes par défaut (comme ORM, authentification, etc.), mais tu peux ajouter des extensions.

Structure du fichier app.py :



```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/")
6  def home():
7      return "Hello from Projet Docker Sécurisé!"
8
9  if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)
11
```

Explication :

- Utilisation de Flask pour créer un serveur web léger.
- Une seule route / pour tester le fonctionnement de l'application.
- Compatible avec une image Docker minimaliste.

3.2. Fonctionnalités de l'application :

Route principale / : affiche un message de bienvenue pour vérifier que l'application fonctionne.

Compatibilité Docker : fonctionne dans l'image distroless Python sans installation supplémentaire.

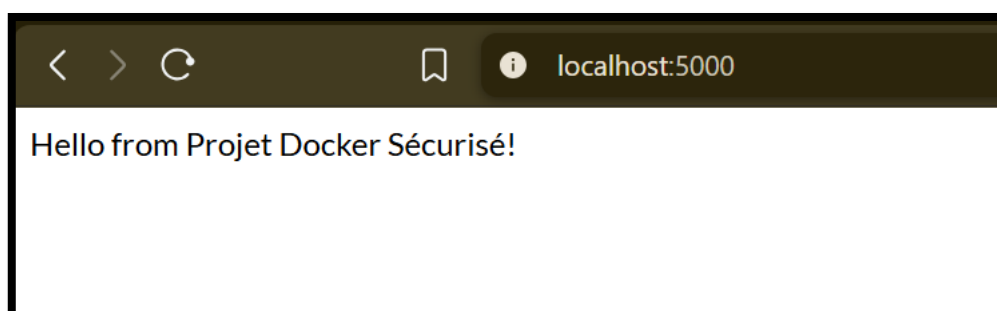
Test de conteneur : permet de vérifier le bon fonctionnement avant la génération de l'image finale.

Ces fonctionnalités sont volontairement minimales pour rester dans le cadre du projet DevSecOps et mettre l'accent sur la sécurisation de l'image Docker plutôt que sur la complexité de l'application.

3.3. Exécution et test local

Test dans le conteneur Docker local :

```
user123@HDM1:/mnt/c/Projet_Docker_Securise$ docker run --rm -it -p 5000:5000 projet_securise
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [03/Dec/2025 18:57:25] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [03/Dec/2025 18:57:25] "GET /favicon.ico HTTP/1.1" 404 -
```



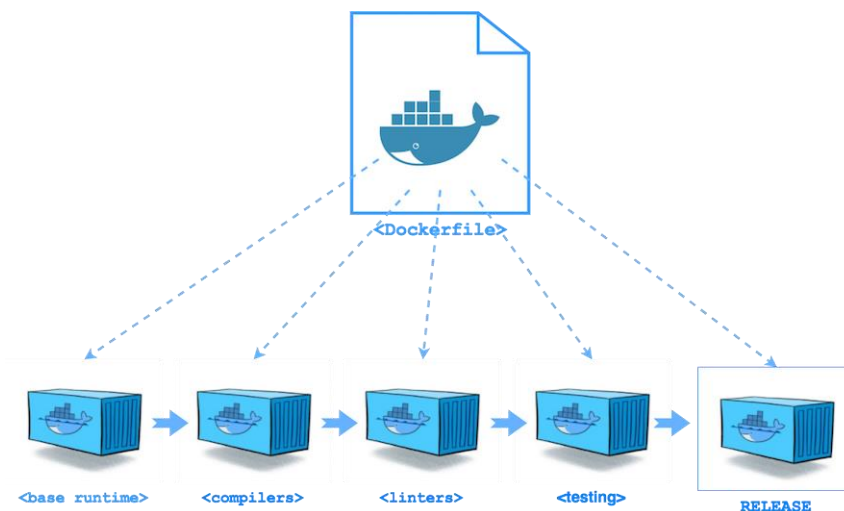
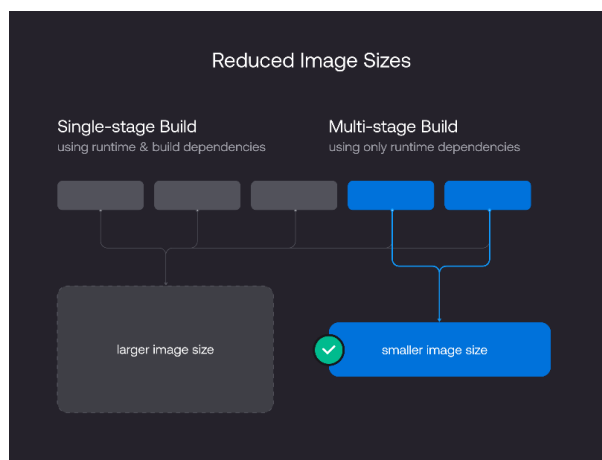
```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker run --rm projet_distroless:latest --version
Python 3.11.2
```

IV. Construction de l'image Docker

4.1. Dockerfile multi-stage build

4.1.1. Définition :

Le multi-stage build est une technique Docker qui permet de construire une image en plusieurs étapes. La première étape (build) compile ou installe les dépendances nécessaires, et la dernière étape (finale) ne contient que ce qui est strictement nécessaire pour exécuter l'application.



Explication :

- L'objectif est de réduire la taille de l'image finale et de supprimer les fichiers temporaires inutiles.

- Permet également de sécuriser l'image en limitant les outils présents dans l'image finale.

```
# -----  
# Étape 1 : Builder (installer les dépendances)  
# -----  
FROM python:3.11-slim AS builder  
WORKDIR /app  
  
# Installer les dépendances dans /app  
COPY App/requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt --target=/app  
  
# Copier le code  
COPY App/ .  
  
# -----  
# Étape 2 : Image finale Distroless non-root  
# -----  
FROM gcr.io/distroless/python3:latest  
WORKDIR /app  
  
# Passer à l'utilisateur non-root fourni par Distroless  
USER nonroot  
  
# Copier le code et dépendances depuis le builder  
COPY --from=builder /app /app  
  
# CMD correct pour Distroless  
CMD ["app.py"]
```

Explications :

- `python:3.11-slim` est l'image minimale avec Python pour construire l'application.
- `--target=/app` permet d'installer les packages directement dans le dossier /app afin de pouvoir les copier dans l'image finale.
- `--no-cache-dir` évite de stocker le cache pip pour réduire la taille.
- `gcr.io/distroless/python3:latest` : image minimale pour exécuter Python, sans shell ni outils supplémentaires.
- `COPY --from=builder /app /app` : copie le code et les dépendances de l'étape builder vers l'image finale.
- `USER nonroot` : exécute l'application avec un utilisateur standard pour plus de sécurité.
- `CMD ["app.py"]` : démarre l'application Python. Distroless ne supporte pas le python en ligne de commande, donc le CMD doit être une liste JSON avec le script principal.

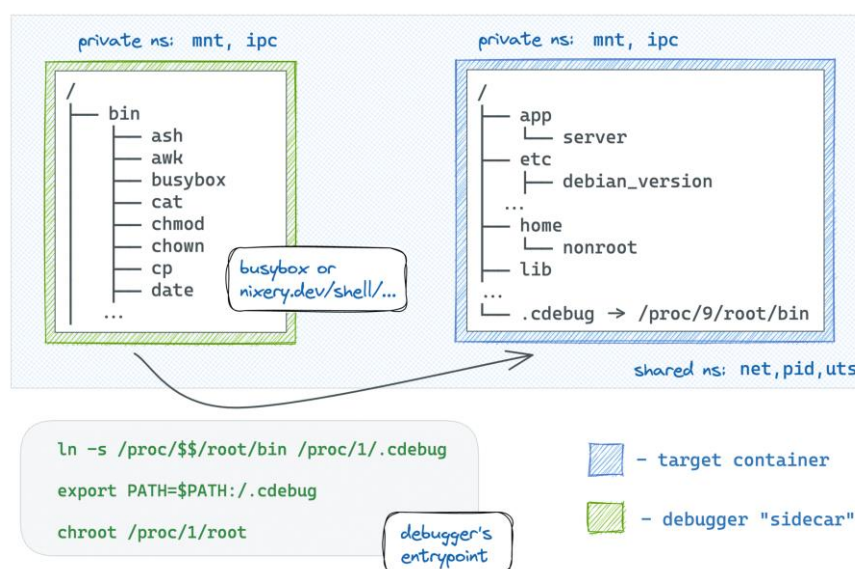
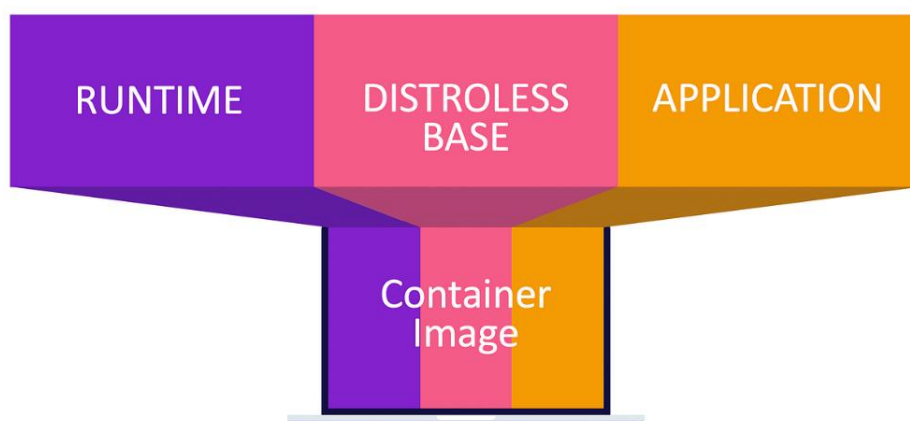
Avantages :

- Réduction de la taille de l'image.
- Limitation des fichiers et dépendances inutiles dans l'image finale.
- Sécurité accrue en réduisant la surface d'attaque.

4.2. Choix de l'image distroless

Définition

Une image Distroless est une image conteneur ultra-minimaliste qui ne contient que mon application et ses dépendances d'exécution, éliminant les outils inutiles comme les gestionnaires de paquets (apt, yum) et les shells (/bin/sh), ce qui réduit considérablement la surface d'attaque, améliore la sécurité et diminue la taille des images pour des déploiements plus rapides.



Caractéristiques principales :

- **Absence d'outils système** : Pas de shell (sh, bash), de gestionnaire de paquets (apt, yum) ou d'utilitaires (curl, wget).
- **Minimalisme** : Contient uniquement l'application et ses bibliothèques d'exécution nécessaires (runtime).
- **Sécurité renforcée** : Surface d'attaque réduite, moins de vulnérabilités (CVEs).
- **Tailles réduites** : Images plus petites, tirages (pulls) plus rapides.
- **Conformité** : Facilite la vérification et la conformité (SBOMs).

Explication :

- Les images distroless réduisent la surface d'attaque car elles n'incluent pas de programmes supplémentaires qui pourraient être exploités.
- Elles permettent également d'alléger l'image finale, ce qui facilite le déploiement et la distribution.

Avantages :

- Plus petite et sécurisée.
- Idéale pour les environnements de production.
- Compatible avec les bonnes pratiques DevSecOps.

La construction de l'image Distroless :

```
user123@HDM1:/mnt/c/Projet_Docker_Securise$ cd /mnt/c/Projet_Docker_Securise/Docker
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker build -t projet_distroless .
[+] Building 17.4s (15/15) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 526B                                0.0s
=> [internal] load metadata for gcr.io/distroless/python3:latest   3.7s
=> [internal] load metadata for docker.io/library/python:3.12-slim 0.4s
=> [internal] load .dockerignore                                    0.1s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                    0.2s
=> => transferring context: 98B                                       0.1s
=> [builder 1/5] FROM docker.io/library/python:3.12-slim@sha256:b43ff04d5df04ad5cabb80890b7ef74e8410e3395b19af97 0.2s
=> => resolve docker.io/library/python:3.12-slim@sha256:b43ff04d5df04ad5cabb80890b7ef74e8410e3395b19af970dcd52d7 0.1s
=> [stage-1 1/4] FROM gcr.io/distroless/python3:latest@sha256:8ce6bba3f793ba7d834467dfe18983c42f9b223604970273e9e3a22b18 10.1s
=> => resolve gcr.io/distroless/python3:latest@sha256:8ce6bba3f793ba7d834467dfe18983c42f9b223604970273e9e3a22b18 0.1s
=> => sha256:069d1e267530c2e681fbd4d481553b4d05f98082b18fafac86e7f12996ddddd0b 131.91kB / 131.91kB 2.1s
=> => sha256:dcaa5a89b0ccda4b283e16d0b4d0891cd93d5fe05c6798f7806781a6a2d84354 314B / 314B 0.8s
=> => sha256:dd64bf2dd177757451a98fcd999a339c35dee5d9872d8f4dc69c8f3c4dd0112 80B / 80B 0.7s
=> => sha256:52630fc75a18675c530ed9eba5f55eca09b03e91bd5bc15307918bbc1a7e7296 162B / 162B 0.7s
=> => extracting sha256:52630fc75a18675c530ed9eba5f55eca09b03e91bd5bc15307918bbc1a7e7296 0.1s
=> => extracting sha256:dd64bf2dd177757451a98fcd999a339c35dee5d9872d8f4dc69c8f3c4dd0112 0.1s
=> => extracting sha256:4aa0ea1413d37a58615488592a0b827ea4b2e48fa5a77cf707d0e35f025e613f 0.1s
=> => extracting sha256:dcaa5a89b0ccda4b283e16d0b4d0891cd93d5fe05c6798f7806781a6a2d84354 0.1s
=> => extracting sha256:069d1e267530c2e681fbd4d481553b4d05f98082b18fafac86e7f12996ddddd0b 0.1s
=> => extracting sha256:e7fa9df358f005850f690ad890b35d30e28b089082a944dc186f5ab3d7f04eb1 0.6s
=> => extracting sha256:d8a0d911b13eb953931298bc7b1e488e278a4a49bcd811fe64205eb29ac4d7b3 0.2s
=> => extracting sha256:5b14f6c9a813108be3fe5634dde0bafbd6e6b35a9929bc6fd2e737f2e618d167 0.1s
=> => extracting sha256:33ce0b1d99fc4503f73b6b587dcb297f1c67cbade50429de5e0517d33e597583 0.1s
=> => extracting sha256:f45e0372ce60322aadbd367c04e72bfce9ee20523b97e8370b844a260efe 0.1s
=> => extracting sha256:7fa8cfa885ce8ddb26aab5406226d134baa65868d7c64f0480cdc88ee0bdcf8 0.1s
=> => extracting sha256:95d5d471a0e3acff350d6c49b1535c25aaddba2abbed47ec53a4651fc5f8aa24 0.1s
=> => extracting sha256:e5770877aff610e4620deb192a20b89b1a718701b990b3f9e2545c7ca8f3987 0.1s
=> => extracting sha256:9e372a664d53bbab62ff716ce57fed5ef99a5fac9ec407f72a9287b9add31dad 0.1s
=> => extracting sha256:6473cd01ad14f8a7ebd42f83bfb483003d99cd163c11c1e3c82c454dce258f06 0.1s
=> => extracting sha256:e6130b4f55d7249b310834e17636fc98f1d0988ba9295212aaf1e02430aadf12 0.1s
```

```

=> => extracting sha256:f793a4be0a15ba5e6a90c4/c3daa/e2c1f78905a505c051b698df345af4e3841 0.1s
=> => extracting sha256:1626fc5dd249961fd0cb9c6f49c438449231cb66ed7a92d7d336c78c98ed0506 0.1s
=> => extracting sha256:588498558164651d733f0e2bf3f67dca89672ec6f1edbbe489d0b1563bf7bbda 0.1s
=> => extracting sha256:02fd24817d0f99f69d0e49e69a5a7ede2be1dcddc91963dce2267a89c6f08627 0.1s
=> => extracting sha256:0101ee5ba170e106c4d4fc6bed9936ab0f3cebe84dcfa5443e437049cef9993 0.1s
=> => extracting sha256:57c7c63c9c91e327147540c9b089813bd33a5ea22bfa8fa617d0636bdb9b5b6 0.1s
=> => extracting sha256:9e0b4d6ceb61f6607f9dda0bcf146143cebec5653ffce6ea1ac6cf19ff66137a 0.1s
=> => extracting sha256:ea95399a5a8dc2319c0e7cbbe39e0ed49517ba5cc4e29c0a1394edc52fda4b2d 0.1s
=> => extracting sha256:e62f3f2af8328754c7763f0310a06c2524f4a9bccaa03dbe9db14917c0055c31 0.1s
=> => extracting sha256:4f4a205960822fb9cda2fc831eea845dd134b58b396a6f3cf11fe0fa58cdfdae 0.1s
=> => extracting sha256:d9e68ac69e7c90abfbb97d0947dfcb6ff6520845e8dbcf4d1bbdbec163a1a4b5 0.1s
=> => extracting sha256:39471dc8c8128839153b809cc07480ad47bcdff6642cbe969adca6b118afe914 0.1s
=> => extracting sha256:c0e669e4287dfc4521862b624ded776f0febab5b2b4ae94ca1085bd0311544d0 0.1s
=> => extracting sha256:5a278cb4e0081c3433a1a5c3d769568a872ead3918e08560230a3b9f26e358da 0.1s
=> => extracting sha256:9039d7835184de356b45040e5d973b3168580a50bd1c711e43cf60895d5d66ee 0.1s
=> => extracting sha256:20004c4a385787f1152afcc092b27a7e4a44859e4cc9d9b65170d16f24619bbe 0.3s
=> => extracting sha256:ee099f293176adf2d765f9675304e7d81e1362247541e90118fcbcc940081c8d 0.5s
=> => extracting sha256:cae86e71460682013795d4c2d230d29b74c7a2163a515f6887b06b91e5f65557 0.3s
=> => extracting sha256:4ebf3752ffbad490f1bcf65fc42a8e6fa8b45098f2b5d4b24e6dfabf49e28dac 0.1s
=> CACHED [builder 2/5] WORKDIR /app 0.0s
=> CACHED [builder 3/5] COPY App/requirements.txt . 0.0s
=> CACHED [builder 4/5] RUN pip install --prefix=/install -r requirements.txt 0.0s
=> CACHED [builder 5/5] COPY App/ . 0.0s
=> [stage-1 2/4] WORKDIR /app 0.2s
=> [stage-1 3/4] COPY --from=builder /install /usr/local 0.7s
=> [stage-1 4/4] COPY --from=builder /app /app 0.1s
=> exporting to image 2.0s
=> => exporting layers 0.9s
=> => exporting manifest sha256:0250c32d9feae99b76ecc334dcff3c648f7f70f6e7bec60f81194608f6622a65 0.0s
=> => exporting config sha256:ec7969cf968e8c32c742143a4ade5001b5c892f47552fe2044b3885c5941b00 0.0s
=> => exporting attestation manifest sha256:9b2f6c565ca964b7297420a44651757ab15651a1f7ee0c1a63d840ec73c2638e 0.1s
=> => exporting manifest list sha256:ac8a4c3551a8860a7187c1de79d6b62a6452a84a6f533ce50b585f5f746ff39 0.0s
=> => naming to docker.io/library/projet_distroless:latest 0.0s
=> => unpacking to docker.io/library/projet_distroless:latest 0.6s

```

```

user123@HDMI:/mnt/c/Projet_Docker_Securise/Docker$ docker history projet_distroless
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
ac8a4c3551a8   27 seconds ago   CMD ["/usr/bin/python3" "app.py"] 0B        buildkit.dockerfile.v0
<missing>      27 seconds ago   COPY /app /app # buildkit          16.4kB    buildkit.dockerfile.v0
<missing>      27 seconds ago   COPY /install /usr/local # buildkit 5.53MB    buildkit.dockerfile.v0
<missing>      28 seconds ago   WORKDIR /app                      8.19kB    buildkit.dockerfile.v0
<missing>      N/A              -                          12.3kB    -
<missing>      N/A              -                          7MB       -
<missing>      N/A              -                          9.2MB     -
<missing>      N/A              -                          5.96MB    -
<missing>      N/A              -                          115kB     -
<missing>      N/A              -                          274kB     -
<missing>      N/A              -                          524kB     -
<missing>      N/A              -                          176kB     -
<missing>      N/A              -                          176kB     -
<missing>      N/A              -                          1.14MB    -
<missing>      N/A              -                          86kB      -
<missing>      N/A              -                          307kB     -
<missing>      N/A              -                          487kB     -
<missing>      N/A              -                          287kB     -
<missing>      N/A              -                          106kB     -
<missing>      N/A              -                          217kB     -
<missing>      N/A              -                          807kB     -
<missing>      N/A              -                          598kB     -
<missing>      N/A              -                          438kB     -
<missing>      N/A              -                          127kB     -
<missing>      N/A              -                          1.73MB    -
<missing>      N/A              -                          385kB     -
<missing>      N/A              -                          451kB     -
<missing>      N/A              -                          1.94MB    -
<missing>      N/A              -                          160kB     -
<missing>      N/A              -                          143kB     -
<missing>      N/A              -                          188kB     -

```



```

user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker inspect hdm17/projet_distroless:latest
[
  {
    "Id": "sha256:ae2e005b3e6ea36d0688c9c3a8c9bcb54a396f9376171818361f04be595523e8",
    "RepoTags": [
      "hdm17/projet_distroless:latest"
    ],
    "RepoDigests": [
      "hdm17/projet_distroless@sha256:ae2e005b3e6ea36d0688c9c3a8c9bcb54a396f9376171818361f04be595523e8"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2025-12-05T01:02:42.756743114Z",
    "Config": {
      "User": "nonroot",
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt",
        "LANG=C.UTF-8"
      ],
      "Entrypoint": [
        "/usr/bin/python3.11"
      ],
      "Cmd": [
        "app.py"
      ],
      "WorkingDir": "/app",
      "ArgsEscaped": true
    },
    "Architecture": "amd64",
    "Os": "linux",
  }
]

```

```

user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker run -it projet_distroless sh
/usr/bin/python3.11: can't open file '/app/sh': [Errno 2] No such file or directory

```

Explication :

Une erreur indiquant que le shell (sh) n'existe pas, ce qui confirme le minimalisme distroless.

4.3. Création d'un utilisateur non-root

Définition :

Un utilisateur non-root est un utilisateur standard sans privilèges d'administrateur. Dans Docker, il est recommandé de ne pas exécuter l'application en tant que root pour limiter les risques en cas de compromission du conteneur.

```

user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker inspect hdm17/projet_distroless:latest
[
  {
    "Id": "sha256:ae2e005b3e6ea36d0688c9c3a8c9bcb54a396f9376171818361f04be595523e8",
    "RepoTags": [
      "hdm17/projet_distroless:latest"
    ],
    "RepoDigests": [
      "hdm17/projet_distroless@sha256:ae2e005b3e6ea36d0688c9c3a8c9bcb54a396f9376171818361f04be595523e8"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2025-12-05T01:02:42.756743114Z",
    "Config": {
      "User": "nonroot",
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt",
        "LANG=C.UTF-8"
      ],
      "Entrypoint": [
        "/usr/bin/python3.11"
      ],
      "Cmd": [
        "app.py"
      ],
      "WorkingDir": "/app",
      "ArgsEscaped": true
    },
    "Architecture": "amd64",
    "Os": "linux",
  }
]

```

Explication :

- Exécuter un conteneur en tant que root peut exposer le système hôte en cas de vulnérabilité.
- Docker permet de créer un utilisateur spécifique dans le Dockerfile pour exécuter l'application.

Avantages :

- Améliore la sécurité du conteneur.
- Réduit le risque de compromission de l'hôte.

4.4. Optimisation de la taille finale

Définition

L'optimisation de l'image Docker consiste à réduire la taille de l'image finale tout en conservant toutes les fonctionnalités nécessaires.

Techniques utilisées :

1. **Multi-stage build** : sépare la phase de build et la phase finale pour ne conserver que l'essentiel.
2. **Suppression des fichiers inutiles** : suppression des caches ou fichiers temporaires après installation.
3. **Choix d'image distroless** : image minimale sans outils superflus.

```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker images hdm17/projet_distroless:latest
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hdm17/projet_distroless:latest	dc7d8527a025	90.8MB	24.5MB	U

V. Publication de l'image sur Docker Hub

5.1. Tag de l'image locale

Définition :

Le tag permet d'identifier une image Docker avec un nom et une version spécifique. Cela facilite la gestion des versions et la publication sur un registre comme Docker Hub.

```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker tag projet_distroless:latest hdm17/projet_distroless:latest
```

Explication :

- `projet_distroless:latest` → nom local de l'image construite.
- `hdm17/projet_distroless:latest` → nom de l'image sur Docker Hub sous mon compte hdm17.

5.2. Push vers Docker Hub

Définition

Le push consiste à envoyer mon image locale vers un registre distant (Docker Hub) afin de la partager ou l'utiliser depuis d'autres environnements.

```
Sélection user123@HDM1: /mnt/c/Projet_Docker_Securise/Docker
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker login
Authenticating with existing credentials... [Username: hdm17]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker push hdm17/projet_distroless:latest
The push refers to repository [docker.io/hdm17/projet_distroless]
687c21efb6ef: Waiting
4aa0ea1413d3: Waiting
1626fc5dd249: Waiting
e62f3f2af832: Waiting
9039d7835184: Waiting
e62f3f2af832: Layer already exists
95d5d471a0e3: Layer already exists
02fd24817d0f: Layer already exists
590f3f180657: Layer already exists
cae86e714606: Layer already exists
d9e68ac69e7c: Layer already exists
2780920e5dbf: Layer already exists
52630fc75a18: Layer already exists
e6130b4f55d7: Layer already exists
```

```

5a278cb4e008: Layer already exists
4ebf3752ffba: Layer already exists
e5770877aff6: Layer already exists
dd64bf2dd177: Layer already exists
0101ee5ba170: Layer already exists
ea95399a5a8d: Layer already exists
069d1e267530: Layer already exists
f793a4be0a15: Layer already exists
5b14f6c9a813: Layer already exists
57c7c63c9c91: Layer already exists
20004c4a3857: Layer already exists
latest: digest: sha256:dc7d8527a02588019ff1e7e1c6bf77b9fd7e82b5e8cdd0e86ffa0db0c7db2ede size: 856

```

Explication :

- **docker login** → authentifie mon compte Docker Hub.
- **docker push** → envoie l'image vers mon dépôt.

5.3. Récupération du digest

Définition

Le digest est un identifiant unique (SHA256) de l'image, utilisé pour vérifier son intégrité et la signer avec des outils comme Cosign.

```

user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker inspect --format='{{index .RepoDigests 0}}' hdm17/projet_distroless:latest
HDM17/projet_distroless@sha256:dc7d8527a02588019ff1e7e1c6bf77b9fd7e82b5e8cdd0e86ffa0db0c7db2ede

```

Explication :

- Le digest permet de s'assurer que l'image poussée est exactement celle qui a été construite localement.
- Il est utilisé pour la signature keyless via Cosign et pour les politiques de sécurité (SLSA 3).

VI. Génération du SBOM

6.1. Rôle et utilité d'un SBOM

Définition :

Un SBOM (Software Bill Of Materials) est une liste complète des composants logiciels présents dans une image ou une application.



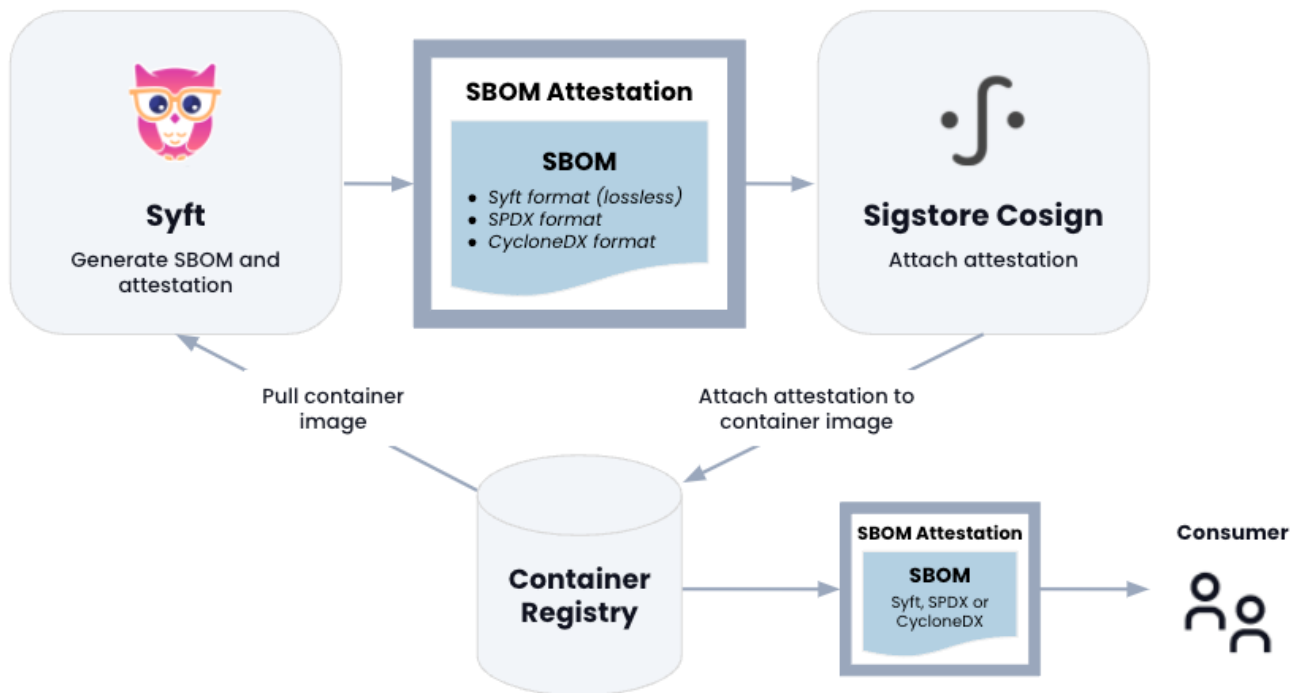
Objectifs :

- Connaître tous les packages et bibliothèques utilisés.
- Identifier rapidement les vulnérabilités connues (CVEs).
- Faciliter la conformité réglementaire et les audits de sécurité.
- Permettre la traçabilité dans la supply chain logiciel (SLSA 3).

6.2. Génération avec Syft

Outil utilisé :

[Syft](#), développé par Anchore.



```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ syft hdm17/projet_distroless:latest -o json > sbom.json
✓Loaded image                                hdm17/projet_distroless:latest
✓Parsed image                                sha256:54e005489429782411487f1aca7916bdc5b20757fb5fbaf4e493b573972eda9d
✓Cataloged contents                          a65178d09d73fb1540a0a23d5c879a04c81470762fd41749cd82b43c80f730d2
├── ✓Packages                                [41 packages]
├── ✓File metadata                            [2,084 locations]
├── ✓Executables                             [357 executables]
└── ✓File digests                            [2,084 files]
```

Explications :

- **hdm17/projet_distroless:latest** → image Docker à analyser.
- **-o json** → format JSON détaillé pour analyses automatisées.

Fichiers générés :

- **sbom.json** → SBOM complet avec toutes les bibliothèques et dépendances.

6.3. Vérification du SBOM

- Objectif : S'assurer que le SBOM contient toutes les dépendances nécessaires et qu'il est conforme à la sécurité attendue.

```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ jq '.artifacts[] | {name: .name, version: .version}' sbom.json
{
  "name": "base-files",
  "version": "12.4+deb12u12"
}
{
  "name": "blinker",
  "version": "1.9.0"
}
{
  "name": "click",
  "version": "8.3.1"
}
{
  "name": "flask",
  "version": "3.1.2"
}
{
  "name": "gcc-12-base",
  "version": "12.2.0-14+deb12u1"
}
```

```
{
  "name": "itsdangerous",
  "version": "2.2.0"
}
{
  "name": "jinja2",
  "version": "3.1.6"
}
{
  "name": "libbz2-1.0",
  "version": "1.0.8-5+b1"
}
{
  "name": "libc6",
  "version": "2.36-9+deb12u13"
}
{
  "name": "libcom-err2",
  "version": "1.47.0-2+b2"
}
{
  "name": "libcrypt1",
  "version": "1:4.4.33-2"
}
```

```
{
  "name": "libk5crypto3",
  "version": "1.20.1-2+deb12u4"
}
{
  "name": "libkeyutils1",
  "version": "1.6.3-2"
}
{
  "name": "libkrb5-3",
  "version": "1.20.1-2+deb12u4"
}
{
  "name": "libkrb5support0",
  "version": "1.20.1-2+deb12u4"
}
{
  "name": "liblzma5",
  "version": "5.4.1-1"
}
{
  "name": "libncursesw6",
  "version": "6.4-4"
}
```

```
{
  "name": "libdb5.3",
  "version": "5.3.28+dfsg2-1"
}
{
  "name": "libexpat1",
  "version": "2.5.0-1+deb12u2"
}
{
  "name": "libffi8",
  "version": "3.4.4-1"
}
{
  "name": "libgcc-s1",
  "version": "12.2.0-14+deb12u1"
}
{
  "name": "libgomp1",
  "version": "12.2.0-14+deb12u1"
}
{
  "name": "libgssapi-krb5-2",
  "version": "1.20.1-2+deb12u4"
}
```

Explication :

- **jq** → filtre les noms et versions des packages dans le SBOM complet pour vérifier l'exhaustivité.

```
{
  "name": "libns12",
  "version": "1.3.0-2"
}
{
  "name": "libpython3.11-minimal",
  "version": "3.11.2-6+deb12u6"
}
{
  "name": "libpython3.11-stdlib",
  "version": "3.11.2-6+deb12u6"
}
{
  "name": "libreadline8",
  "version": "8.2-1.3"
}
{
  "name": "libsqlite3-0",
  "version": "3.40.1-2+deb12u2"
}
{
  "name": "libssl3",
  "version": "3.0.17-1~deb12u3"
}
```

```
{
  "name": "libstdc++6",
  "version": "12.2.0-14+deb12u1"
}
{
  "name": "libtinfo6",
  "version": "6.4-4"
}
{
  "name": "libtirpc3",
  "version": "1.3.3+ds-1"
}
{
  "name": "libuuid1",
  "version": "2.38.1-5+deb12u3"
}
{
  "name": "markupsafe",
  "version": "3.0.3"
}
{
  "name": "media-types",
  "version": "10.0.0"
}
```

```
{
  "name": "netbase",
  "version": "6.4"
}
{
  "name": "python3-distutils",
  "version": "3.11.2-3"
}
{
  "name": "python3.11-minimal",
  "version": "3.11.2-6+deb12u6"
}
{
  "name": "tzdata",
  "version": "2025b-0+deb12u2"
}
{
  "name": "werkzeug",
  "version": "3.1.4"
}
{
  "name": "zlib1g",
  "version": "1:1.2.13.dfsg-1"
}
user123@HDM1: /mnt/c/Projet_Docke
```

VII. Analyse de sécurité de l'image

L'analyse de sécurité d'une image Docker consiste à rechercher des vulnérabilités connues (CVE) dans les paquets et dépendances composant l'image. Dans ce projet nous utilisons Gype (Anchore) pour effectuer ces scans.

7.1. Scan avec Gype (mode offline)

Définition

Gype est un outil de scanning d'images Docker qui analyse :

- Le système de base (OS packages)
- Les dépendances applicatives (Python, Node, Go...)
- Les bibliothèques embarquées

En mode offline, Grype utilise la base de vulnérabilités locale (Syft + DB locale), ce qui garantit qu'aucune donnée n'est envoyée à Internet mais que la base doit être à jour.

Il détecte :

- Les vulnérabilités (CVEs),
- Leur sévérité (Critical, High, Medium, Low),
- Et les packages concernés.

Mode offline

Il existe deux manières d'utiliser Grype :

1. Mode online → utilise la base de données vulnérabilités d'Anchore, mise à jour automatiquement.
2. Mode offline → utilise une base locale téléchargée par l'utilisateur.

Je vais utiliser le mode offline, recommandé dans les contextes sécurisés.

```
^Cuser123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ grype hdm17/projet_distroless:latest -o table
✓Vulnerability DB [updated]
✓Loaded image
✓Parsed image
✓Cataloged contents
├─ ✓Packages [41 packages]
├─ ✓Executables [357 executables]
├─ ✓File metadata [2,084 locations]
├─ ✓File digests [2,084 files]
└─ ✓Scanned for vulnerabilities [62 vulnerability matches]
    └─ by severity: 1 critical, 4 high, 16 medium, 9 low, 32 negligible
    └─ by status: 0 fixed, 62 not-fixed, 0 ignored (1 dropped)[0001] WARN unable to parse additio
```

Explications :

- **hdm17/projet_distroless:latest** → image Docker à scanner.
- **-o table** → résultat sous forme lisible par un humain.

7.2. Vulnérabilités détectées

Après l'exécution du scan, Gripe fournit une liste de vulnérabilités triées par sévérité.

NAME	INSTALLED	FIXED IN	TYPE	VULNERABILITY	SEVERITY	EPSS	RISK
libpython3.11-minimal	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-8194	High	0.2% (44th)	0.2
libpython3.11-stdlib	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-8194	High	0.2% (44th)	0.2
python3.11-minimal	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-8194	High	0.2% (44th)	0.2
libexpat1	2.5.0-1+deb12u2	(won't fix)	deb	CVE-2025-59375	High	0.1% (33rd)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2018-20796	Negligible	1.8% (82nd)	< 0.1
libsqlite3-0	3.40.1-2+deb12u2	(won't fix)	deb	CVE-2025-7458	Critical	< 0.1% (25th)	< 0.1
libpython3.11-minimal	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-6069	Medium	0.2% (37th)	< 0.1
libpython3.11-stdlib	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-6069	Medium	0.2% (37th)	< 0.1
python3.11-minimal	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-6069	Medium	0.2% (37th)	< 0.1
libpython3.11-minimal	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-8291	Medium	0.1% (30th)	< 0.1
libpython3.11-stdlib	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-8291	Medium	0.1% (30th)	< 0.1
python3.11-minimal	3.11.2-6+deb12u6	(won't fix)	deb	CVE-2025-8291	Medium	0.1% (30th)	< 0.1
libexpat1	2.5.0-1+deb12u2		deb	CVE-2024-28757	Negligible	0.9% (75th)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2019-1010023	Negligible	0.7% (71st)	< 0.1
libsqlite3-0	3.40.1-2+deb12u2	(won't fix)	deb	CVE-2025-7709	Medium	< 0.1% (18th)	< 0.1
libncursesw6	6.4-4	(won't fix)	deb	CVE-2023-50495	Medium	< 0.1% (15th)	< 0.1
libtinfo6	6.4-4	(won't fix)	deb	CVE-2023-50495	Medium	< 0.1% (15th)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2010-4756	Negligible	0.5% (65th)	< 0.1
libpython3.11-minimal	3.11.2-6+deb12u6		deb	CVE-2025-13836	Medium	< 0.1% (13th)	< 0.1
libpython3.11-stdlib	3.11.2-6+deb12u6		deb	CVE-2025-13836	Medium	< 0.1% (13th)	< 0.1
python3.11-minimal	3.11.2-6+deb12u6		deb	CVE-2025-13836	Medium	< 0.1% (13th)	< 0.1
libgssapi-krb5-2	1.20.1-2+deb12u4		deb	CVE-2018-5709	Negligible	0.5% (63rd)	< 0.1
lib5crypto3	1.20.1-2+deb12u4		deb	CVE-2018-5709	Negligible	0.5% (63rd)	< 0.1
libkrb5-3	1.20.1-2+deb12u4		deb	CVE-2018-5709	Negligible	0.5% (63rd)	< 0.1
libkrb5support0	1.20.1-2+deb12u4		deb	CVE-2018-5709	Negligible	0.5% (63rd)	< 0.1
libpython3.11-minimal	3.11.2-6+deb12u6		deb	CVE-2025-12084	Medium	< 0.1% (11th)	< 0.1
libpython3.11-stdlib	3.11.2-6+deb12u6		deb	CVE-2025-12084	Medium	< 0.1% (11th)	< 0.1
python3.11-minimal	3.11.2-6+deb12u6		deb	CVE-2025-12084	Medium	< 0.1% (11th)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2019-1010024	Negligible	0.4% (58th)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2019-9192	Negligible	0.4% (56th)	< 0.1
libsqlite3-0	3.40.1-2+deb12u2		deb	CVE-2021-45346	Negligible	0.2% (47th)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2019-1010025	Negligible	0.2% (45th)	< 0.1
libgssapi-krb5-2	1.20.1-2+deb12u4		deb	CVE-2024-26458	Negligible	0.2% (42nd)	< 0.1
lib5crypto3	1.20.1-2+deb12u4		deb	CVE-2024-26458	Negligible	0.2% (42nd)	< 0.1
libkrb5-3	1.20.1-2+deb12u4		deb	CVE-2024-26458	Negligible	0.2% (42nd)	< 0.1
libkrb5support0	1.20.1-2+deb12u4		deb	CVE-2024-26458	Negligible	0.2% (42nd)	< 0.1
libc6	2.36-9+deb12u13		deb	CVE-2019-1010022	Negligible	0.1% (35th)	< 0.1
libncursesw6	6.4-4	(won't fix)	deb	CVE-2025-6141	Low	< 0.1% (2nd)	< 0.1
libtinfo6	6.4-4	(won't fix)	deb	CVE-2025-6141	Low	< 0.1% (2nd)	< 0.1

Analyse des résultats Gripe :

- Total vulnérabilités détectées : 62
- Critical : 1
- High : 4
- Medium : 16
- Low : 9
- Negligible : 32

Interprétation commune :

Dans les images distroless, il est courant :

- d'avoir très peu de vulnérabilités,
- souvent Low ou Negligible,
- parfois aucune si ton runtime Python est propre.

7.3. Gestion et interprétation des résultats

Objectif

Comprendre ce que signifient les vulnérabilités détectées et comment les gérer dans une démarche de supply chain sécurisée (SLSA).

Interprétation :

- **Critical / High** → doivent être corrigées immédiatement (mettre à jour le package, changer d'image de base).
- **Medium** → surveiller ; corriger si facile.
- **Low / Negligible** → souvent liées à des métadonnées ou dépendances non dangereuses.
- **Unknown** → vulnérabilité signalée mais non classée.

Comment les corriger ?

- **Régénérer l'image avec une base plus récente :**
 - ✓ `docker pull gcr.io/distroless/python3:latest`
- **Mettre à jour les dépendances Python :**
 - ✓ `pip install -U flask`
- **Reconstruire l'image :**
 - ✓ `docker build -t hdm17/projet_distroless:latest`
- **Scanner à nouveau pour confirmer.**

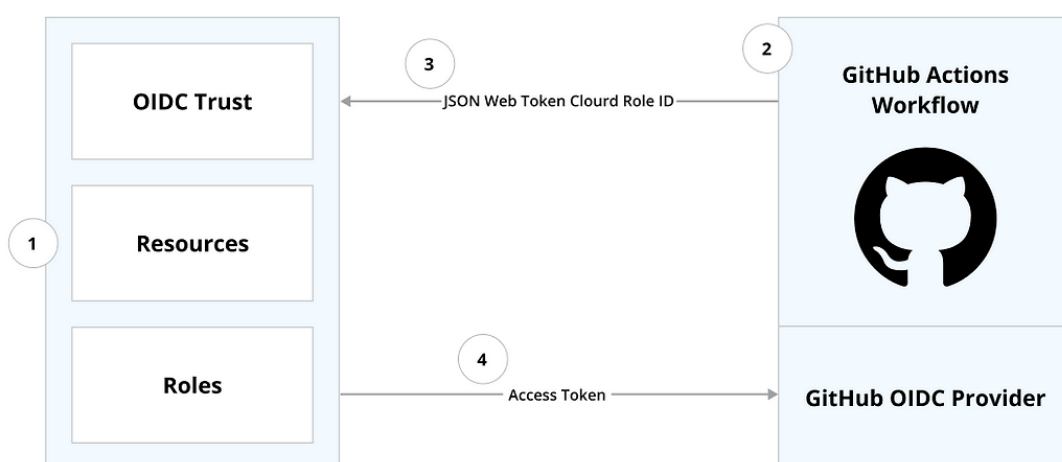
VIII. Signature de l'image Docker

8.1. Présentation de la signature keyless (OIDC)

Définition :

La signature keyless permet de signer une image Docker sans générer ni stocker une clé privée locale. Elle utilise le protocole OIDC (OpenID Connect) pour authentifier l'utilisateur via un fournisseur (ex. Docker Hub, GitHub, Google).

JFrog Platform



Objectif :

- Garantir l'intégrité et l'authenticité de l'image.
- Assurer la traçabilité dans la chaîne de confiance (*supply chain*) SLSA 3.
- Éviter la gestion des clés locales et réduire les risques de compromission.

8.2. Connexion Cosign / Docker Hub

Pré-requis : Docker Hub actif et connecté dans Docker Desktop.

```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ docker login
Authenticating with existing credentials... [Username: hdm17]
Info → To login with a different account, run 'docker logout' followed by 'docker login'
Login Succeeded
```

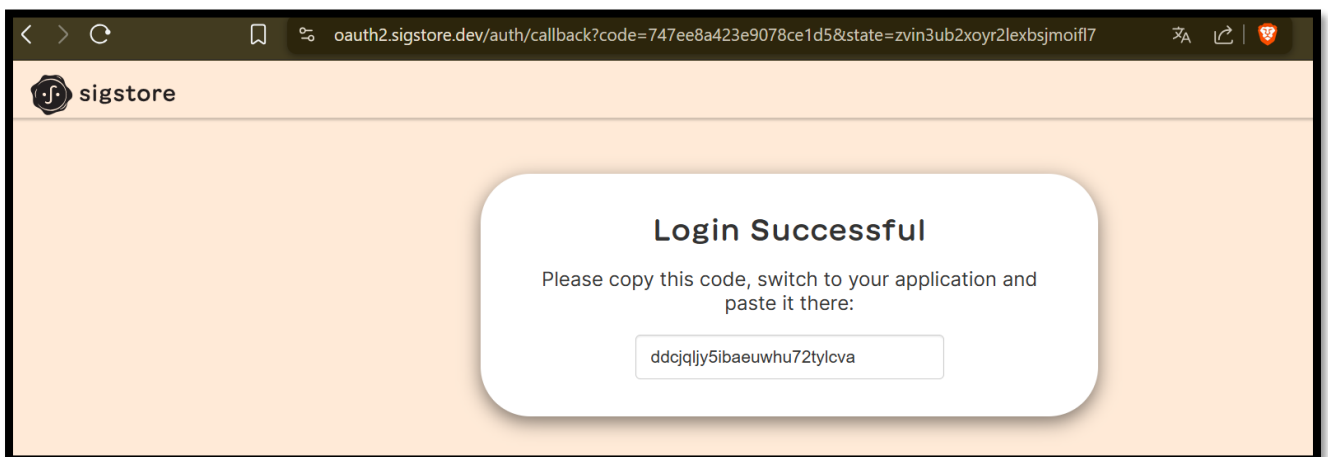
8.3. Signature de l'image

```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ cosign sign docker.io/hdm17/projet_distroless:latest
WARNING: Image reference docker.io/hdm17/projet_distroless:latest uses a tag, not a digest, to identify the image to sign.
This can lead you to sign a different image than the intended one. Please use a
digest (example.com/ubuntu@sha256:abc123...) rather than tag
(example.com/ubuntu:latest) for the input to cosign. The ability to refer to
images by tag will be removed in a future release.

The sigstore service, hosted by sigstore a Series of LF Projects, LLC, is provided pursuant to the Hosted Project Tools Terms of Use, available at https://lfproject
s.org/policies/hosted-project-tools-terms-of-use/.
Note that if your submission includes personal data associated with this signed artifact, it will be part of an immutable record.
This may include the email address associated with the account with which you authenticate your contractual Agreement.
This information will be used for signing this artifact and will be stored in public transparency logs and cannot be removed later, and is subject to the Immutable
Record notice at https://lfprojects.org/policies/hosted-project-tools-immutable-records/.

By typing 'y', you attest that (1) you are not submitting the personal data of any other person; and (2) you understand and agree to the statement and the Agreement terms a
t the URLs listed above.
Are you sure you would like to continue? [y/N] y
error opening browser: exec: "xdg-open": executable file not found in $PATH
Go to the following link in a browser:

https://oauth2.sigstore.dev/auth/auth?access_type=online&client_id=sigstore&code_challenge=7GFA8a_rB9FFXGvvL7HzdjPLvqPrTk0ghECHVtsowr4&code_challenge_method=S256&n
once=36UCjmHlu7c4cXzBysE6ehn0sN0&redirect_uri=urn%3Aietf%3Awg%3Aoauth%3A2.0%3Aaob&response_type=code&scope=openid+email&state=36UCjpmJGIq6tC243DsUQgnAH2b
Enter verification code: ddcjqljy5ibaeuwhu72tylcva
```



Explication :

- `docker.io/hdm17/projet_distroless:latest` → image à signer.
- Cosign génère une signature et l'enregistre dans le registre Docker Hub.

Résultat attendu :

- Signature attachée à l'image.
- Pas besoin de clé locale grâce au mode keyless.

8.4. Vérification de la signature (SLSA 3 compliant)

`--certificate-identity "..."` → on précise l'identité attendue (mon email GitHub).

`--certificate-oidc-issuer "https://github.com/login/oauth"` → le serveur OIDC utilisé pour authentifier le signataire.

`docker.io/hdm17/projet_distroless:latest` → image Docker à vérifier.

```
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$ cosign verify --certificate-identity "houssameddinemarchoual@gmail.com" \
> --certificate-oidc-issuer "https://github.com/login/oauth" \
> docker.io/hdm17/projet_distroless:latest

Verification for index.docker.io/hdm17/projet_distroless:latest --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The code-signing certificate was verified using trusted certificate authority certificates

[{"critical":{"identity":{"docker-reference":"index.docker.io/hdm17/projet_distroless:latest"},"image":{"docker-manifest-digest":"sha256:e82b5e8cdd0e86ffa0db0c7db2ede"},"type":"https://sigstore.dev/cosign/sign/v1"},"optional":null}]
user123@HDM1:/mnt/c/Projet_Docker_Securise/Docker$
```

Explication :

- ✓ **Cosign claims validated** → La signature contenue dans l'image correspond à mon identité et aux métadonnées attendues.
- ✓ **Existence in the transparency log verified offline** → La signature est bien enregistrée dans le Sigstore transparency log, ce qui garantit qu'elle est publique et traçable.
- ✓ **Code-signing certificate verified** → Le certificat utilisé pour signer l'image est reconnu et fiable (émis par une autorité de confiance).
- ✓ **docker-reference** → référence de l'image Docker signée.
- ✓ **docker-manifest-digest** → l'empreinte unique (hash SHA256) de l'image. Cela garantit que l'image que je vérifie est exactement celle qui a été signée.
- ✓ **type** → format de signature utilisé par Cosign.

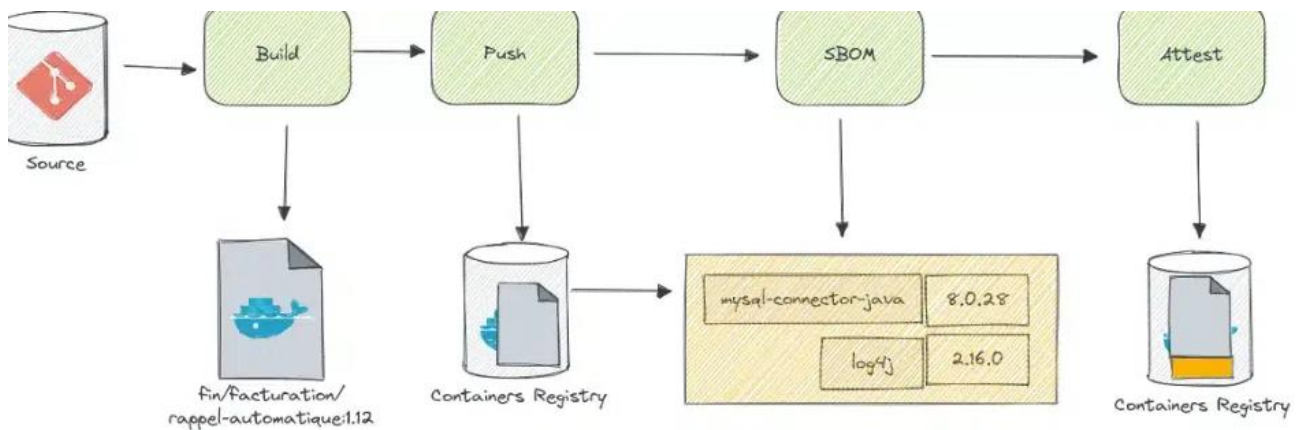
Mon image est correctement signée avec Cosign en mode keyless, et la signature est valide, traçable et conforme SLSA 3.

X. Conclusion

9.1. Résumé des étapes

Au cours de ce projet, nous avons réalisé l'ensemble des étapes suivantes :

1. **Prérequis et environnement**
 - Installation de Windows 11 + WSL Ubuntu pour disposer d'un environnement Linux complet.
 - Configuration de Docker Desktop, Python, BuildKit, Syft, Cosign et Gype.
2. **Développement de l'application**
 - Création d'une application Flask minimaliste (app.py) avec affichage d'un message simple.
 - Test local pour confirmer que l'application fonctionne correctement.
3. **Construction de l'image Docker**
 - Création d'un Dockerfile multi-stage pour séparer la phase de build et l'image finale.
 - Utilisation d'une image distroless pour réduire la taille et la surface d'attaque.
 - Création d'un utilisateur non-root pour renforcer la sécurité.
4. **Publication sur Docker Hub**
 - Tag de l'image locale et push vers le registre Docker Hub.
 - Récupération du digest pour garantir l'immuabilité de l'image.
5. **Génération du SBOM (Software Bill Of Materials)**
 - Utilisation de Syft pour générer sbom.json et sbom_summary.json.
 - Vérification de l'exhaustivité des dépendances et bibliothèques utilisées.
6. **Analyse de sécurité de l'image**
 - Scan avec Gype pour détecter les vulnérabilités.
 - Interprétation des résultats et mise à jour des dépendances pour corriger les vulnérabilités critiques et élevées.
7. **Signature de l'image Docker**
 - Authentification via Docker Hub et OIDC GitHub.
 - Signature keyless de l'image avec Cosign.
 - Vérification de la signature, assurant la traçabilité et la conformité SLSA 3.



9.2. Avantages du modèle DevSecOps et pistes d'amélioration

Avantages :

- Sécurité intégrée dès le début : Les vulnérabilités sont détectées et corrigées avant la mise en production.
- Automatisation et traçabilité : Grâce aux outils comme Cosign, Syft et Grype, les étapes de build, scan et signature sont traçables et reproductibles.
- Images optimisées et minimalistes : Utilisation de distroless réduit la surface d'attaque et le temps de déploiement.
- Conformité et audits facilités : SBOM et signatures garantissent la transparence pour les audits et la supply chain.

Pistes d'amélioration :

- Mettre en place des CI/CD pipelines automatisés pour le build, scan et signature.
- Ajouter des tests unitaires et de sécurité plus complets dans l'image Docker.
- Surveiller régulièrement les vulnérabilités des dépendances avec des mises à jour automatiques.
- Intégrer des outils complémentaires comme in-toto pour une chaîne de confiance complète.